



**Patrones de software**

**Implementación de patrones v5**

**Docente: ELIECER MONTERO OJEDA**

**Ángel David Reyes Moreno**

**UNIDADES TECNOLÓGICAS DE SANTANDER  
Ingeniería de Sistemas  
Bucaramanga, 2026**

# PATRONES CREACIONALES

## Patrón 1: Singleton

¿Qué problema resuelve? Garantiza que solo exista una única instancia de un objeto durante toda la vida de la aplicación. Útil para configuración y conexiones a bases de datos.

```
export class ConfigManager {  
  // 1. Variable estática que guarda LA única instancia  
  private static instance: ConfigManager | null = null;  
  
  // 2. Constructor PRIVADO → nadie puede hacer "new ConfigManager()"   
  private constructor() {  
    this.config = { /* Lee las variables de entorno */ }  
  }  
  
  // 3. Punto de acceso global → siempre devuelve la misma instancia  
  public static getInstance(): ConfigManager {  
    if (!ConfigManager.instance) {  
      ConfigManager.instance = new ConfigManager(); // solo se crea UNA vez  
    }  
    return ConfigManager.instance; // las demás veces, devuelve la existente  
  }  
}
```

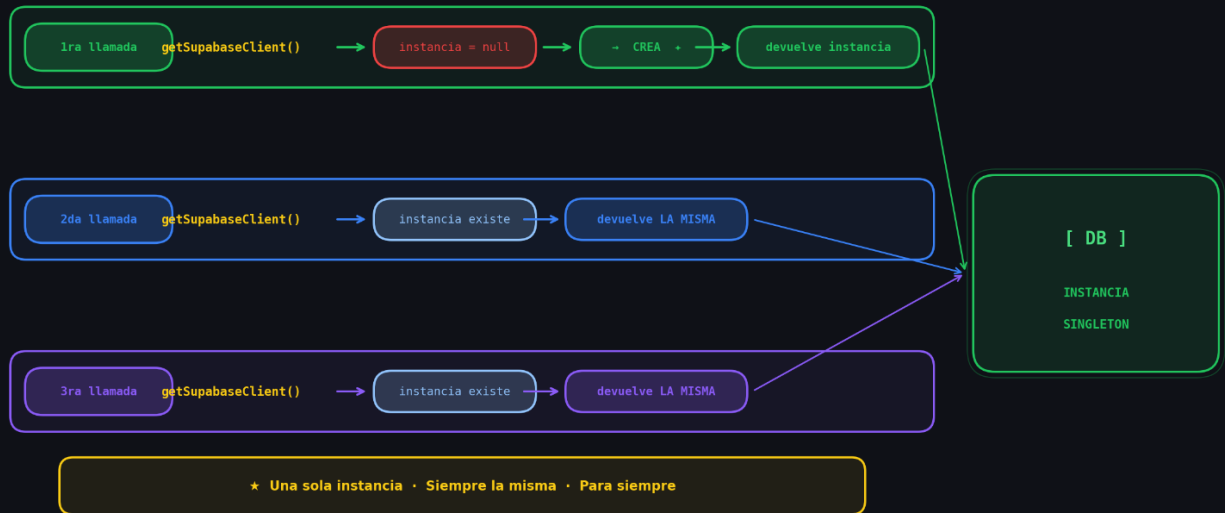
¿Dónde se usa? El cliente de Supabase lo llama al iniciarse:

```
let supabaseInstance: SupabaseClient | null = null;  
  
export function getSupabaseClient(): SupabaseClient {  
  if (supabaseInstance) return supabaseInstance; // ya existe → devuelve la misma  
  
  const config = ConfigManager.getInstance(); // + usa el otro Singleton  
  supabaseInstance = createClient(config.get("supabaseUrl"), ...);  
  return supabaseInstance;  
}
```

Diagrama Uml

## PATRÓN SINGLETON

getSupabaseClient()



## Patrón 2: Factory Method

¿Qué problema resuelve? Delegar la **creación de objetos** a subclases, sin que el código cliente sepa qué clase concreta se está creando. Perfecto para múltiples tipos de dispositivos IoT.

```
// Creator abstracto define el "molde"
abstract class DeviceAdapterCreator {
  abstract createAdapter(id: string, type: DeviceType): IDeviceAdapter; // ← Factory Method
}

// Cada subclase decide QUÉ adaptador concreto crear
class SimulatedAdapterCreator extends DeviceAdapterCreator {
  createAdapter(id: string, type: DeviceType): IDeviceAdapter {
    return new SimulatedDeviceAdapter(id, type); // concreta
  }
}

// Punto de entrada público — elige el Creator correcto
export class DeviceAdapterFactory {
  static async create(params): Promise<IDeviceAdapter> {
    switch (params.protocol) {
      case "MQTT": return new MQTTAdapterCreator().getAndConnect(...);
      case "REST": return new RESTAdapterCreator().getAndConnect(...);
      case "SIMULATED": return new SimulatedAdapterCreator().getAndConnect(...);
    }
  }
}
```

¿Dónde se usa en la UI?

La UI solo llama a la fábrica. De momento se está instanciando un protocolo de simulación si en un futuro se implementa un protocolo en tiempo real solo hay que crear ese protocolo ejemplo: ZIGBEE, solo ZigbeeDeviceAdapter y ZigbeeAdapterCreator — el código de la UI no cambia para nada seguirá llamando a la fábrica .

```
// La página NO sabe qué adaptador concreto se crea
const adapter = await DeviceAdapterFactory.create({
  deviceId: "dev-001",
  deviceType: "SOLAR_PANEL",
  protocol: "SIMULATED", // ← aquí solo cambia el protocolo
});
const reading = await adapter.readData(); // misma interfaz siempre
```

Diagrama Uml

# FACTORY METHOD — Device Adapter

Patron de disenio creacional

«interface»

**IDeviceAdapter**

connect() · readData() · disconnect()

Contrato de comunicacion con dispositivos

implementa

implementaciones concretas

**MQTTDeviceAdapter**

connect()

(fisica real)

**RESTDeviceAdapter**

connect()

(API HTTP)

**SimulatedDeviceAdapter**

connect()<sup>crea / usa</sup>

(datos sinteticos)

«abstract»

**DeviceAdapterCreator**

createAdapter() ~ Factory Method

subclases deben implementar createAdapter()

extiende

creators concretos

**MQTTAdapterCreator**

createAdapter()

retorna instancia concreta

**RESTAdapterCreator**

createAdapter()

retorna instancia concreta

**SimulatedAdapterCreator**

createAdapter()

retorna instancia concreta

★ Cada Creator sabe cual Adapter crear — el cliente solo habla con IDeviceAdapter

### Patrón 3: Abstract Factory

#### ¿Qué problema resuelve?

En la plataforma tenemos **4 fuentes de energía** (Solar, Eólica, Batería, Red). Cada una necesita tres componentes que **deben ser coherentes entre sí**:

- Un lector de producción
- Un calculador de precios
- Un modelo de predicción

Sin el patrón, la UI tendría que saber qué clase concreta instanciar en cada caso, generando código espagueti

#### ¿Dónde se usa?

Se define las tres faces del producto

```
interface IEnergyProducer { getCurrentReading(): Promise<EnergyReading>; }
interface IPriceEstimator { getCurrentQuote(): Promise<PriceQuote>; }
interface IForecastModel { getDailyForecast(): Promise<DailyForecast>; }
```

Se define la Abstract Factory (el contrato de la fábrica)

```
interface IEnergySourceFactory {
  createProducer(): IEnergyProducer; // + crea producto A
  createPriceEstimator(): IPriceEstimator; // + crea producto B
  createForecastModel(): IForecastModel; // + crea producto C
}
```

Se implementan las 4 fábricas concretas, una por familia

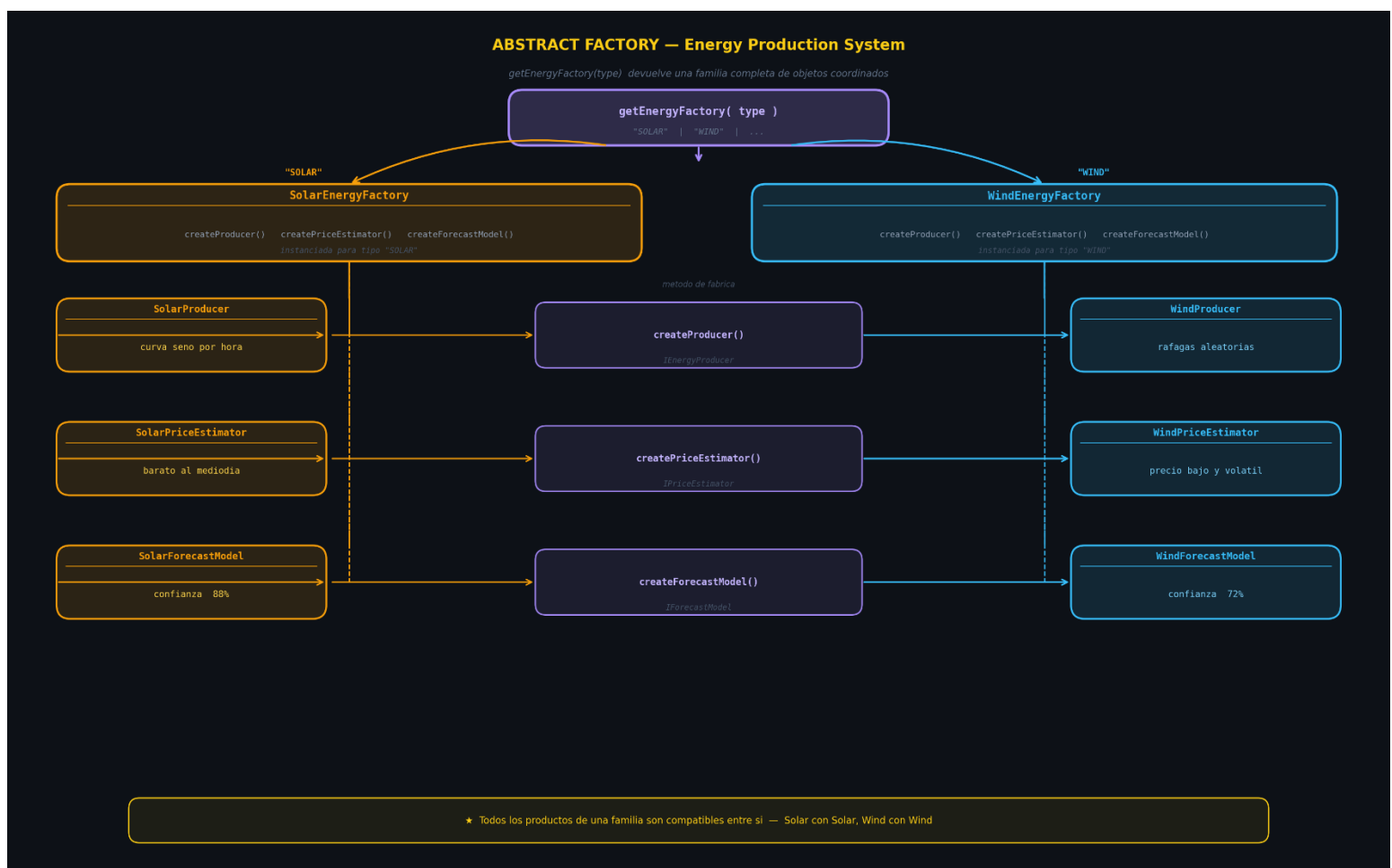
```
class SolarEnergyFactory implements IEnergySourceFactory {
  createProducer() { return new SolarProducer(); } // curva seno solar
  createPriceEstimator() { return new SolarPriceEstimator(); } // más barato al mediodía
  createForecastModel() { return new SolarForecastModel(); } // confianza 88%
}

class WindEnergyFactory ... // variación tipo ráfaga, confianza 72%
class BatteryEnergyFactory ... // carga/descarga, confianza 95%
class GridEnergyFactory ... // ilimitada, la más cara, confianza 99%
```

Por ultimo el selector de las cuatro clases de fabrica

```
export function getEnergyFactory(source: EnergySource): IEnergySourceFactory {  
  switch (source) {  
    case "SOLAR": return new SolarEnergyFactory();  
    case "WIND": return new WindEnergyFactory();  
    case "BATTERY": return new BatteryEnergyFactory();  
    case "GRID": return new GridEnergyFactory();  
  }  
}
```

## Diagrama Uml



## Patrón 4: Builder

### ¿Qué problema resuelve?

Una TradeOrder tiene muchos campos opcionales. Sin Builder, el constructor se convierte en un desastre:

```
// ✗ Sin Builder - imposible saber qué significa cada argumento  
new TradeOrder("SELL", 12.5, 0.124, "SOLAR", "FIXED", "URGENT", 30, true, false, "Excedente")
```

Con Builder se construye la orden **paso a paso**, solo con lo que se necesita :

```
new TradeOrderBuilder()  
  .ofType("SELL")  
  .withAmount(12.5)  
  .atPrice(0.124)  
  .fromSource("SOLAR")  
  .withPriority("URGENT")  
  .build()
```

### ¿Dónde se usa?

El patrón tiene **3 participantes** hasta el momento en el proyecto

#### 1. El Producto — TradeOrder (interfaz)

El objeto complejo que se construye al final:

```
interface TradeOrder {  
  id: string;          type: OrderType;      amountKwh: number;  
  pricePerKwh: number; energySource: EnergySource;  
  pricingMode: PricingMode; priority: OrderPriority;  
  expiresAt: Date | null; note: string | null;  
  conditions: TradeOrderConditions;  
  maxSlippagePercent: number; totalValueUsd: number;  
  status: OrderStatus;  
}
```



## 2. El Builder — TradeOrderBuilder

Acumula el estado internamente y expone métodos encadenables:

```
class TradeOrderBuilder {
  // campos privados con valores por defecto seguros
  private _type: OrderType | null = null;
  private _amountKwh: number | null = null;
  private _priority: OrderPriority = "NORMAL"; // default
  private _conditions: TradeOrderConditions = {};
  // ...

  ofType(type)      { this._type = type;      return this; }
  withAmount(kwh)   { this._amountKwh = kwh;   return this; }
  atPrice(usd)      { this._pricePerKwh = usd;  return this; }
  fromSource(src)   { this._energySource = src; return this; }
  expiresInMinutes(min) { /* calcula Date */    return this; }
  requireGreenCertified() { this._conditions.requireGreenCertified = true; return this; }

  build(): TradeOrder {
    // Valida campos requeridos
    if (!this._type) throw new TradeOrderBuildError("El tipo es requerido");
    if (this._pricePerKwh <= 0) throw new TradeOrderBuildError("El precio debe ser > 0");
    // Calcula campos derivados y retorna el producto final
    return { id: `ORD-...`, totalValueUsd: kwh * price, status: "DRAFT", ... };
  }

  reset() { /* Limpia todo para reutilizar */ return this; }
}
```

## 3. El Director — TradeOrderDirector

Encapsula **recetas predefinidas** usando el Builder. Sabe qué pasos llamar y en qué orden:

```
class TradeOrderDirector {
  constructor(private builder: TradeOrderBuilder) {}

  buildUrgentSolarSell(kwh, price): TradeOrder {
    return this.builder.reset()
      .ofType("SELL").withAmount(kwh).atPrice(price)
      .fromSource("SOLAR").withPricingMode("DYNAMIC")
      .withPriority("URGENT").expiresInMinutes(30)
      .withMaxSlippage(5).allowPartialFill()
      .build();
  }

  buildScheduledGreenBuy(kwh, maxPrice): TradeOrder { ... }
  buildPeakBatteryDischarge(kwh, price): TradeOrder { ... }
}
```

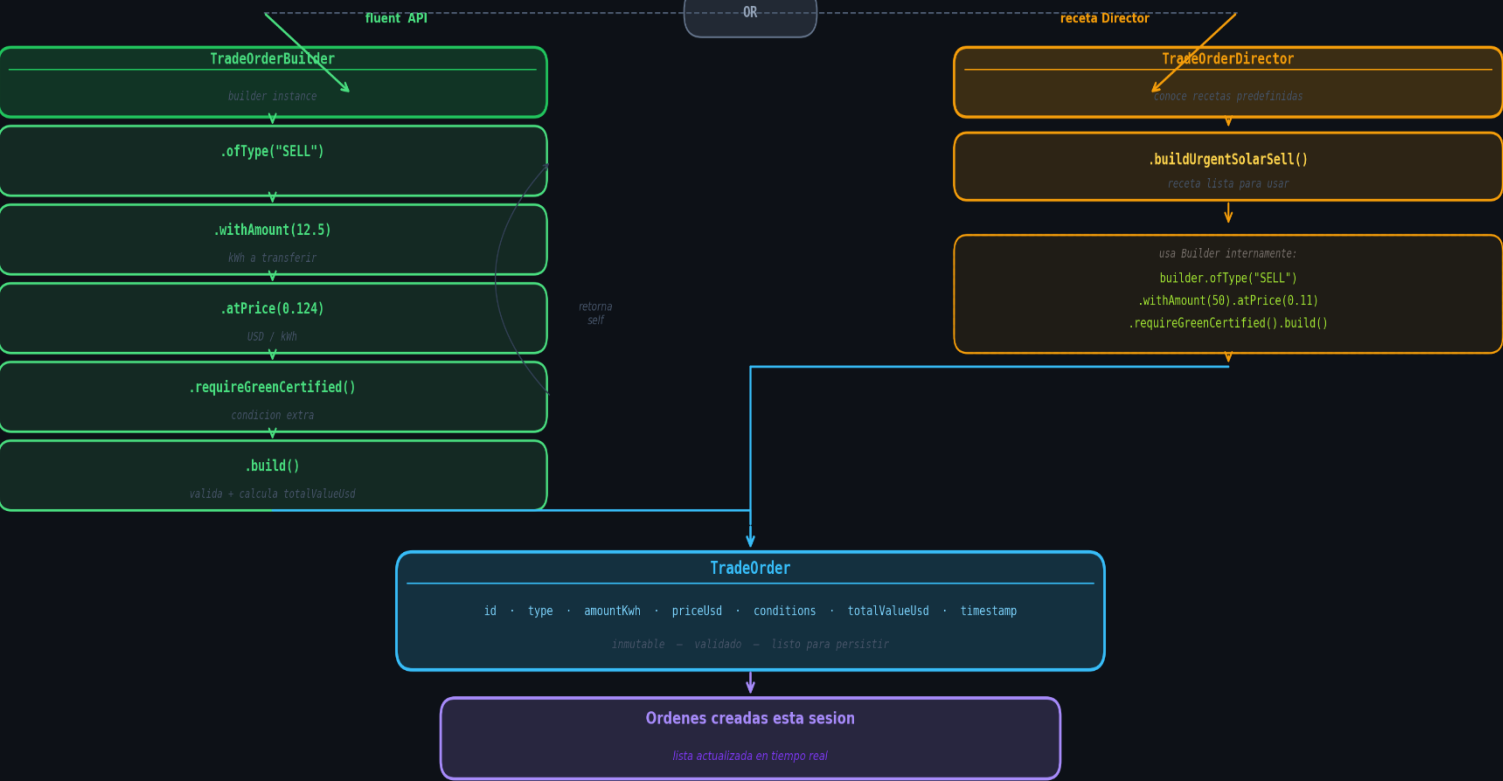
Diagrama Uml

PATRON BUILDER — TradeOrder Construction

Dos rutas para construir la misma orden — fluent API o Director con receta

Usuario llena el formulario / hace clic en receta

campos: tipo, cantidad, precio, condiciones



LEYENDA

Usuario / entrada

Builder (fluent API)

Director (receta)

Producto final: TradeOrder

UI / vista

★ El Director es opcional — simplifica la creacion de objetos complejos con configuraciones frecuentes

## Patrón 5: Prototype

### ¿Qué problema resuelve?

Un trader vende 10 kWh solares cada mañana. Sin Prototype, lo reconstruye con el Builder desde cero todos los días. Con Prototype, guarda esa orden como plantilla y la clona en segundos el clon es 100% independiente, tiene nuevo ID y puede pisar campos.

### ¿Dónde se usa?

```
interface IOrderPrototype {  
  clone(): OrderTemplate;  
  getOrder(): TradeOrder;  
}
```

```
class OrderTemplate implements IOrderPrototype {  
  private readonly _order: TradeOrder; // orden congelada (immutable)  
  
  clone(overrides: OrderOverrides = {}): OrderTemplate {  
    this.usageCount++;  
  
    // Reconstruye con el Builder → nuevo ID, validación garantizada  
    const builder = new TradeOrderBuilder()  
      .ofType(this._order.type)  
      .withAmount(overrides.amountKwh ?? this._order.amountKwh) // ← override opcional  
      .atPrice(overrides.pricePerKwh ?? this._order.pricePerKwh)  
      .fromSource(this._order.energySource)  
      // ... copia todas las condiciones del original  
      .withNote(`Clon de: ${this.name}] ${this._order.note}`);  
  
    return new OrderTemplate(builder.build(), `${this.name} (copia)`);  
  }  
}
```

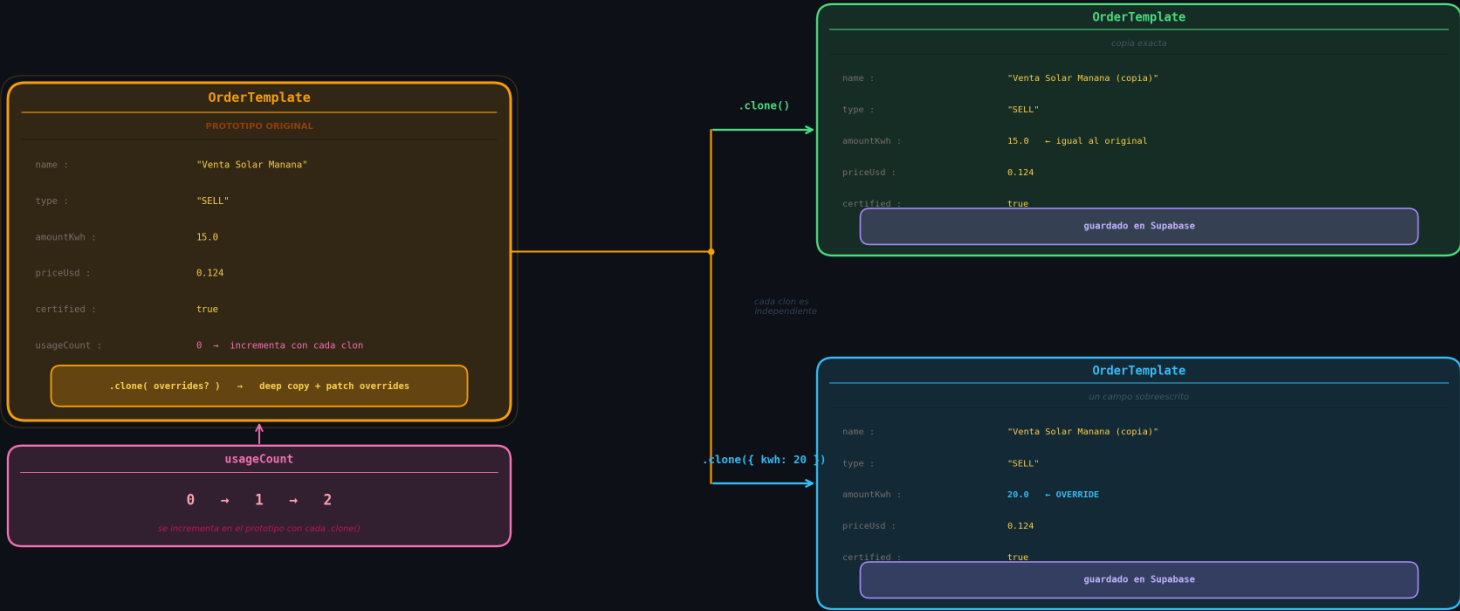
Registra las plantillas y las manda a la base de datos de supabase

```
class OrderTemplateRegistry {  
  // 3 templates pre-cargados: Venta Solar Mañana, Carga Batería Noche, Compra Verde  
  
  cloneFrom("Venta Solar Mañana", { amountKwh: 15 })  
  // → nueva TradeOrder: 15 kWh (override), precio/fuente/condiciones del original  
}
```

# Diagrama Uml

## PATRON PROTOTYPE — OrderTemplate Cloning

Crea nuevas ordenes copiando una plantilla existente — sin reescribir todos los campos



★ El prototipo nunca se modifica — cada `.clone()` produce un objeto independiente listo para persistir

# PATRONES ESTRUCTURALES

## Patrón 1: Adapter

### ¿Qué problema resuelve?

el Adapter traduce la interfaz incompatible de cada proveedor externo a la interfaz estándar IPriceEstimator. El resto del sistema (páginas, servicios) solo conoce IPriceEstimator y no sabe nada del proveedor real.

### ¿Dónde se usa?

**1.Target:** la interfaz que el sistema usa internamente:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>; // → siempre USD/kWh  
}
```

**2 Adaptees:** las interfaces incompatibles de cada proveedor:

```
interface IOmieMarketApi {  
  fetchDayAheadPrice(fecha: string): Promise<{  
    precio_mwh: number; // ← €/MWh (no USD/kWh)  
    tecnologia: "SOLAR" | "EOLICA" | "HIDRO" | "TERMICO"; // ← español  
    variacion_pct: number;  
  }>;  
}
```

**3 Los Adapters:** traducen el formato externo al interno:

```

class OmieAdapter implements IPriceEstimator { // + implementa Target
  constructor(private omieApi: IOmieMarketApi) {} // + envuelve al Adaptee

  async getCurrentQuote(): Promise<PriceQuote> {
    const raw = await this.omieApi.fetchDayAheadPrice(today);

    // TRADUCCIÓN: €/MWh → USD/kWh
    const priceUsdKwh = raw.precio_mwh * 1.08 * 0.001;

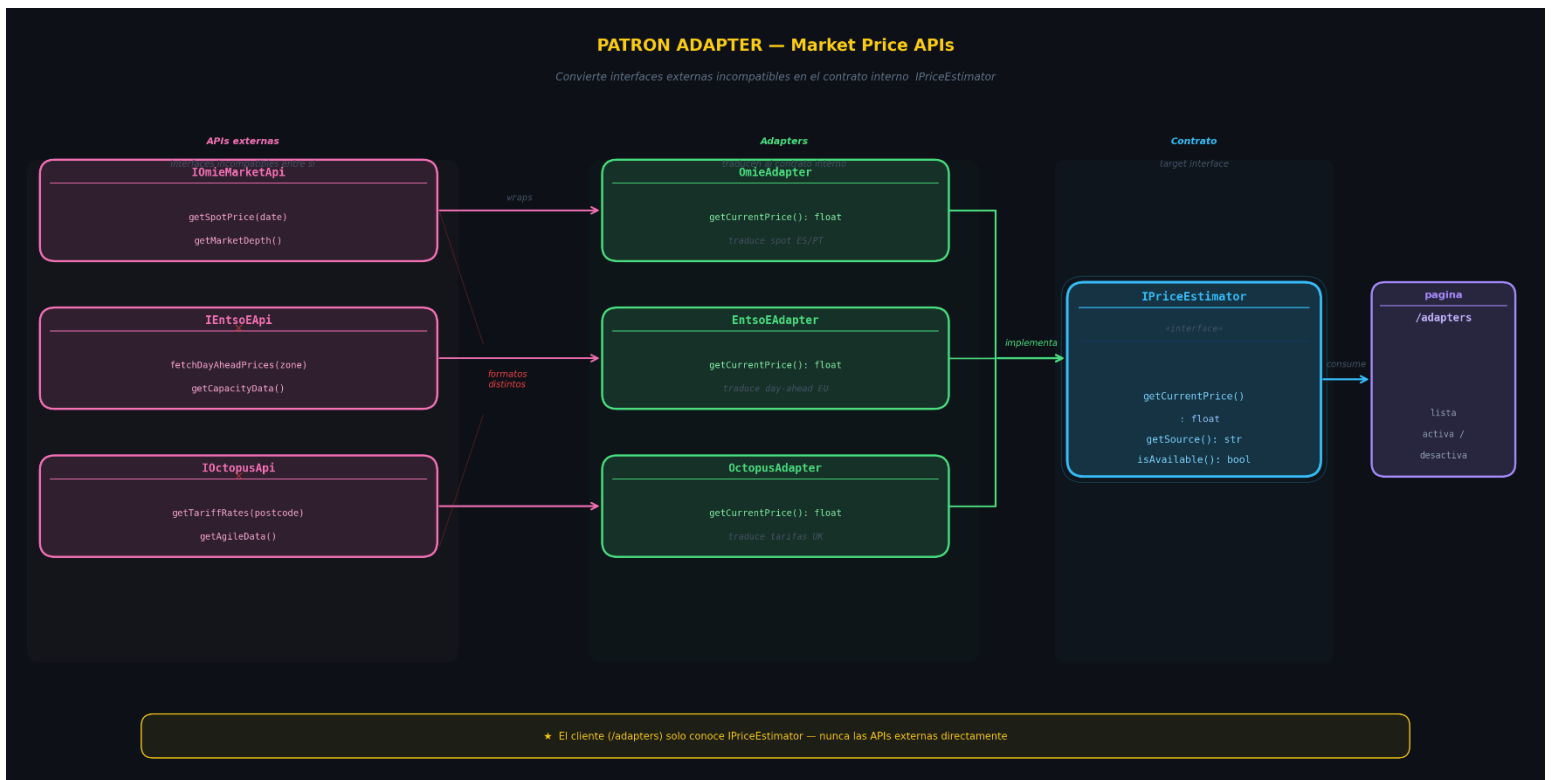
    // TRADUCCIÓN: "EOLICA" → "WIND"
    const source = { SOLAR:"SOLAR", EOLICA:"WIND", ... }[raw.tecnologia];

    // TRADUCCIÓN: variacion_pct → "UP"/"DOWN"/"STABLE"
    const trend = raw.variacion_pct > 1 ? "UP" : ...

    return { source, pricePerKwh: priceUsdKwh, trend, currency: "USD", ... };
  }
}

```

## Diagrama Uml



## Patrón 2: Bridge

### ¿Qué problema resuelve?

el sistema genera notificaciones de 3 tipos distintos (precio, orden, dispositivo) y necesita entregarlas por múltiples canales (consola, in-app, email, webhook). Sin Bridge, habría  $3 \times 4 = 12$  clases Con Bridge:  $3 + 4 = 7$  clases

### ¿Dónde se usa?

**IMPLEMENTOR** — la jerarquía de cómo se entrega:

```
interface INotificationChannel {  
    deliver(notification: Notification): Promise<void>;  
}  
  
class ConsoleChannel implements INotificationChannel { ... } //  
imprime en terminal  
class InAppChannel implements INotificationChannel { ... } //  
guarda en memoria → UI  
class EmailChannel implements INotificationChannel { ... } //  
simula SendGrid  
class WebhookChannel implements INotificationChannel { ... } //  
simula POST HTTP
```

**ABSTRACTION** — la clase base que compone el canal (no hereda):

```
abstract class Notifier {  
    constructor(protected channel: INotificationChannel) {} // ← el  
    "bridge"  
  
    setChannel(channel: INotificationChannel) { // cambia canal en  
        runtime  
        this.channel = channel;  
    }  
}
```

**REFINED ABSTRACTIONS** — la jerarquía de qué se notifica:



```

class PriceAlertNotifier extends Notifier {
  async alertPriceSpike(source, price, threshold) {
    // formatea el mensaje específico de precio...
    await this.channel.deliver({ title: "Precio alto", level:
"CRITICAL", ... });
  }
}

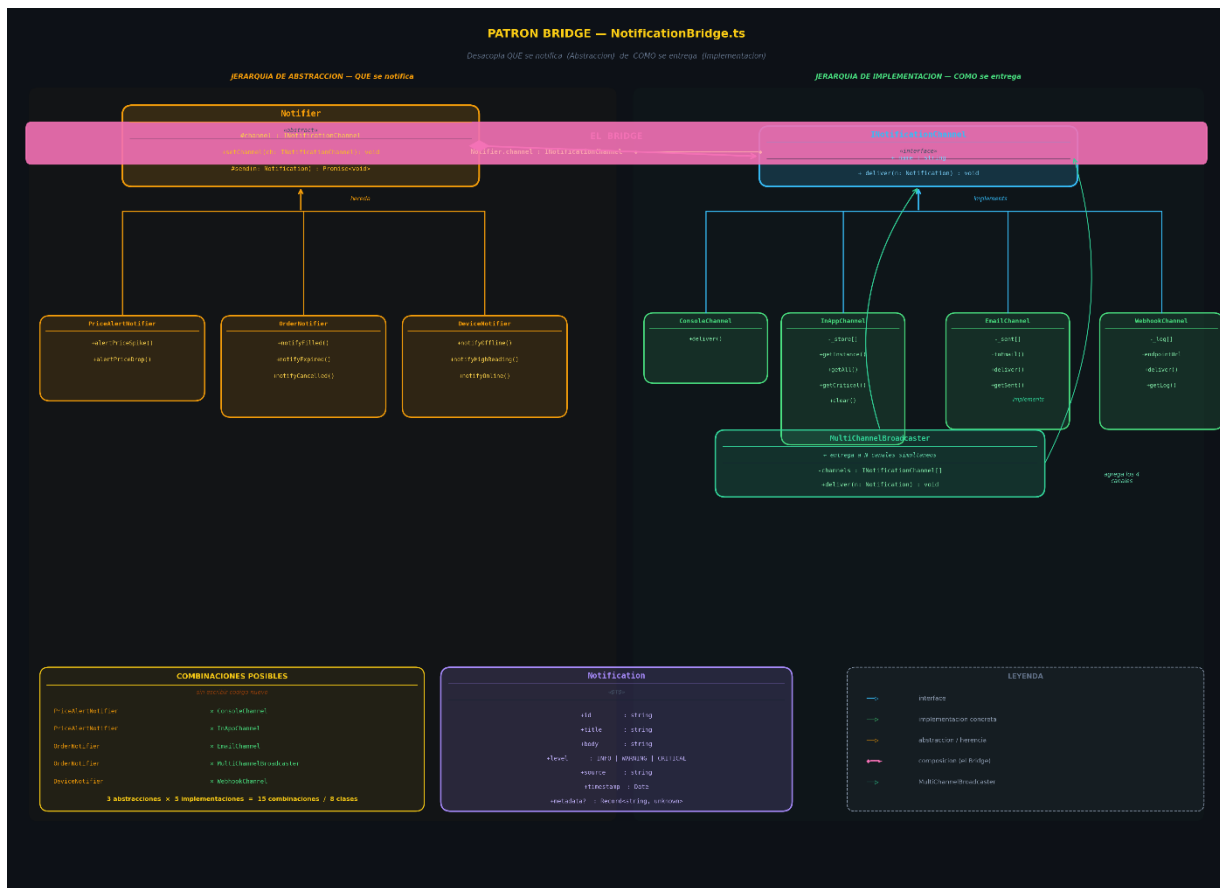
class OrderNotifier extends Notifier {
  async notifyOrderFilled(orderId, kwh, price) { ... }
  async notifyOrderExpired(orderId, reason) { ... }
}

class DeviceNotifier extends Notifier {
  async notifyDeviceOffline(deviceId, name) { ... }
}

```

El "Bridge" es la referencia **protected channel** en la clase base. Al cambiar el canal, todo el comportamiento de entrega cambia sin tocar ninguna de las 3 clases de notificación:

## Diagrama Uml



### Patrón 3: Decorator

#### ¿Qué problema resuelve?

El OmieAdapter (del patrón Adapter) obtiene precios, pero falta:

- ¿Cómo evitar llamar la API 60 veces por minuto? → **Caché**
- ¿Cómo saber si los precios subieron de golpe? → **Alertas**
- ¿Cómo registrar cada consulta para análisis? → **Logging**
- ¿Qué pasa si la red falla? → **Reintentos**

Sin Decorator: habría que meter todo eso dentro del OmieAdapter, mezclando responsabilidades. Con Decorator: **cada comportamiento es una capa independiente y apilable.**

#### ¿Dónde se usa?

**COMPONENT** — la interfaz del patrón Adapter:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>;  
}
```

**DECORATOR BASE** — envuelve cualquier IPriceEstimator:

```
abstract class PriceEstimatorDecorator implements IPriceEstimator {  
  constructor(protected readonly wrapped: IPriceEstimator) {}  
  
  getCurrentQuote(): Promise<PriceQuote> {  
    return this.wrapped.getCurrentQuote(); // delega por defecto  
  }  
}
```

**4 DECORADORES CONCRETOS** — cada uno añade una sola responsabilidad:

```

class CachingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    if (cache valida) return this._cached; // corto-circuito
    const quote = await this.wrapped.getCurrentQuote(); // delega y cachea
    this._cached = quote;
    return quote;
  }
  getStats() → { hits, misses, ratio }
}

class LoggingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const start = Date.now();
    const quote = await this.wrapped.getCurrentQuote();
    this._history.unshift({ price, durationMs, fromCache, ... });
    return quote;
  }
}

class AlertingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const quote = await this.wrapped.getCurrentQuote();
    if (quote.pricePerKwh > this.spikeThreshold) {
      // USA EL BRIDGE para entregar la alerta + integración entre patrones
      new PriceAlertNotifier(InAppChannel.getInstance()).alertPriceSpike(...);
    }
    return quote;
  }
}

class RetryDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    for (let i = 1; i <= maxAttempts; i++) {
      try { return await this.wrapped.getCurrentQuote(); }
      catch { await sleep(baseDelay * 2^i); } // backoff exponencial
    }
  }
}

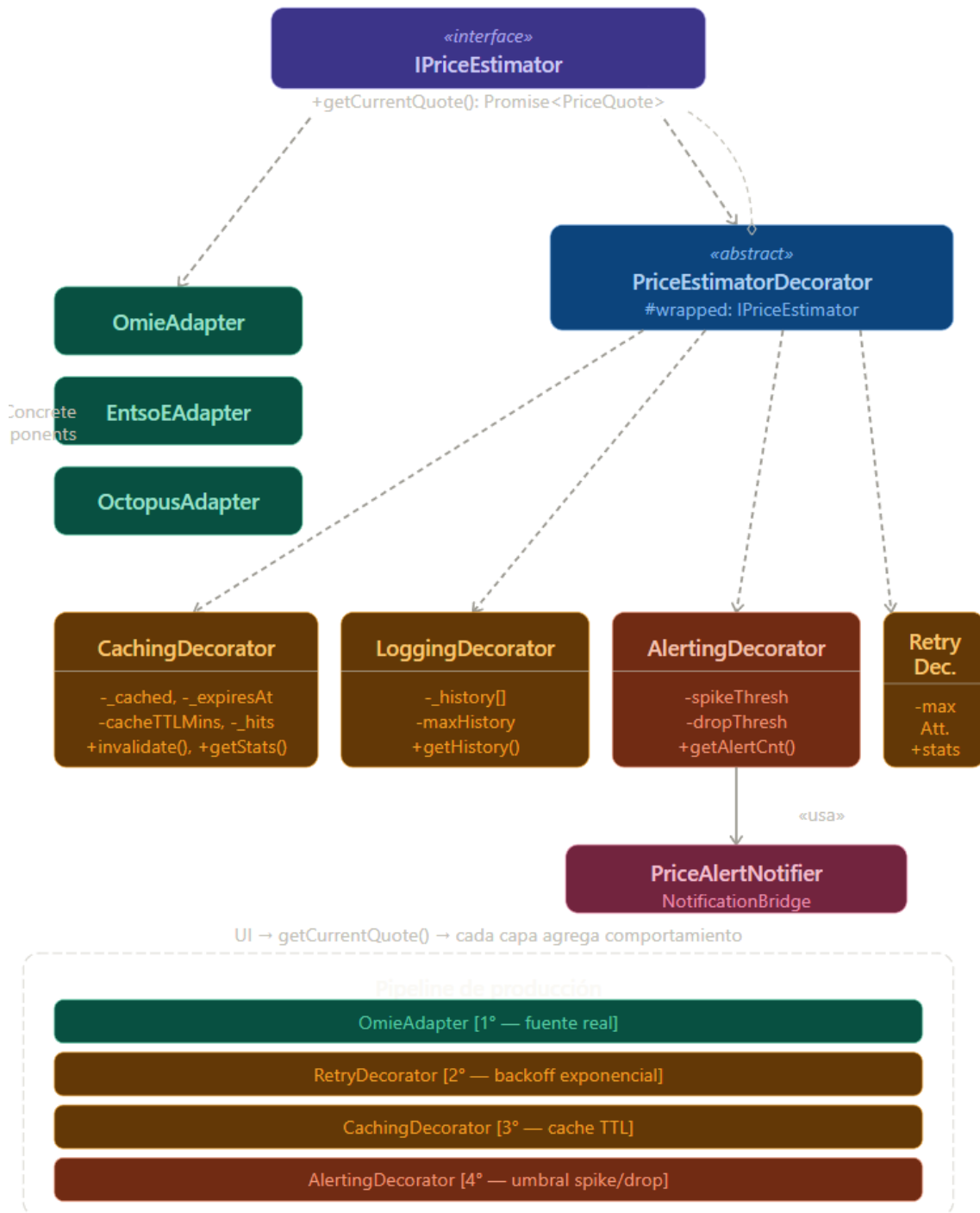
```

**Pipeline completo** — los decoradores se apilan de adentro hacia afuera:

```
const pipeline = buildPricePipeline(new OmieAdapter(api), {
  cacheTTLMinutes: 1,
  spikeThreshold: 0.18,
  maxRetries: 3,
});

// Capa más externa → Logging
//   → Alerting
//     → Caching
//       → Retry
//         → OmieAdapter (capa más interna)
```

## Diagrama Uml



## Patrón 4: Facade

### ¿Qué problema resuelve?

Antes de la Facade, para que una página cree y publique una orden necesitaba conocer y coordinar 5 subsistemas por separado.

```
// ❌ Sin Facade - el cliente debe conocer TODOS los subsistemas:
const builder = new TradeOrderBuilder(); // subsistema 1
const order = builder.ofType("SELL")...build();
const db = getSupabaseAdmin(); // subsistema 2
await db.from("energy_orders").insert({...});
const notifier = new OrderNotifier(InAppChannel.getInstance()); // subsistema 3
await notifier.notifyOrderFilled(order.id, ...);
```

EnergyTradingFacade expone operaciones de alto nivel que internamente coordinan todos los subsistemas. El cliente solo llama a la Facade y obtiene el resultado.

### ¿Dónde se usa?

La Facade expone 5 operaciones de alto nivel, cada una orquestando subsistemas distintos:

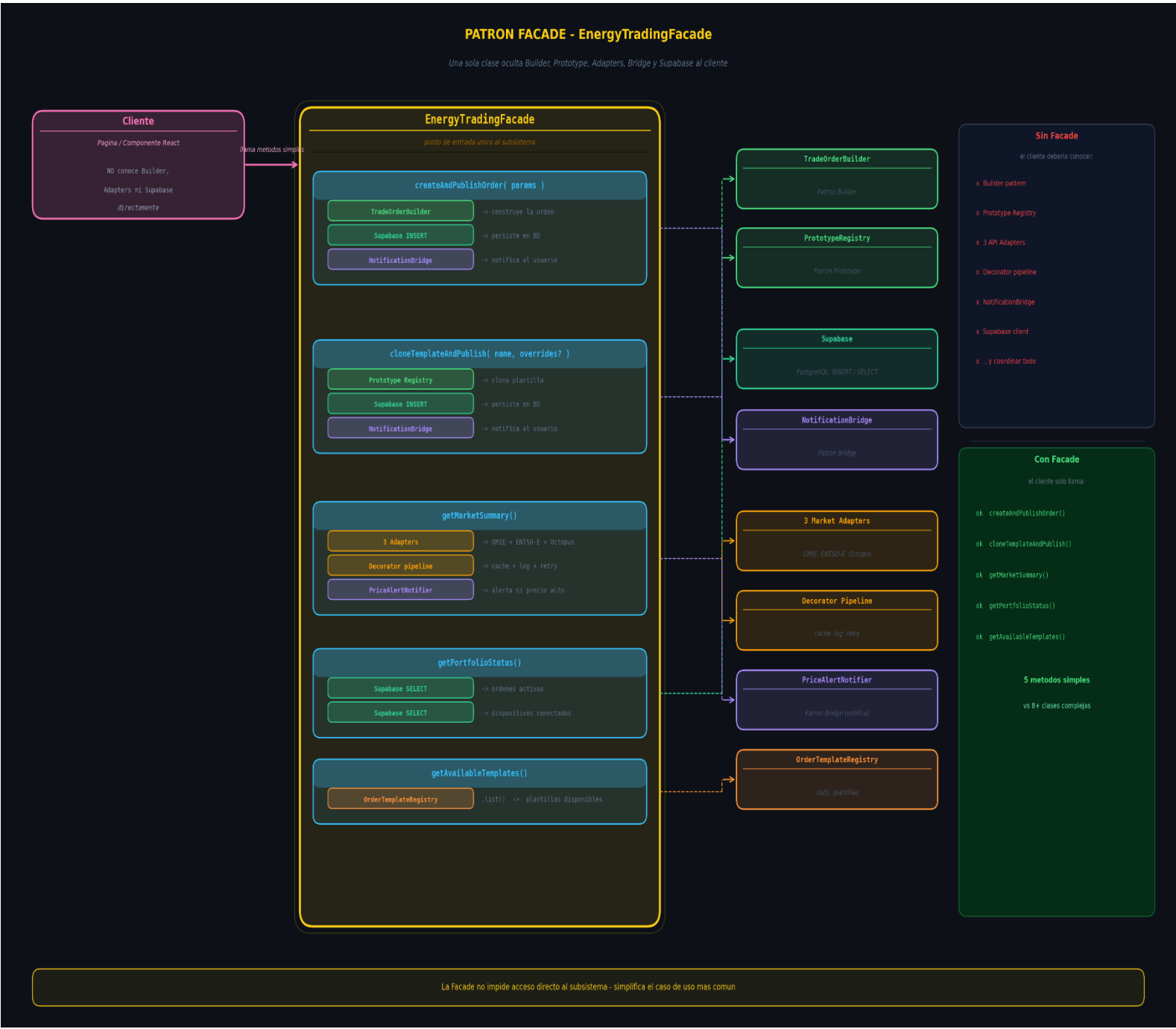
```
async createAndPublishOrder(params: CreateOrderParams): Promise<FacadeResult<string>> {
  // Subsistema 1 - Builder
  const order = new TradeOrderBuilder().ofType(params.type)...build();

  // Subsistema 2 - Supabase
  const { error } = await getSupabaseAdmin().from("energy_orders").insert({...});

  // Subsistema 3 - Bridge de notificaciones
  await new OrderNotifier(InAppChannel.getInstance()).notifyOrderFilled(...);

  return { ok: true, data: order.id };
}
```

Diagrama Uml



## Patrón 5: composite

### ¿Qué problema resuelve?

Una red de energía tiene dispositivos individuales y grupos de dispositivos. Sin Composite, el código tiene que preguntar constantemente "¿es un dispositivo o es un grupo?"

```
// ❌ Sin Composite - Lógica diferente para cada caso:
function calcularTotal(nodo: unknown): number {
  if (nodo instanceof EnergyDevice) {
    return nodo.energyKwh; // caso hoja
  } else if (nodo instanceof EnergyGroup) {
    return nodo.children.reduce((sum, c) => sum + calcularTotal(c), 0); // caso grupo
  }
  return 0;
}
```

Con Composite, ambos implementan IEnergyNode y el cliente llama siempre lo mismo

```
// ✅ Con Composite - el cliente no sabe si es hoja o grupo:
unPanel.getTotalKwh() // → 42.5 kWh
unaGranja.getTotalKwh() // → 91.8 kWh (suma recursiva de 3 paneles)
elPortfolio.getTotalKwh() // → 593.8 kWh (suma de toda la jerarquía)
```

### ¿Dónde se usa?

**Component** — la interfaz común

```
interface IEnergyNode {
  getTotalKwh(): number; // energía total
  getPeakPowerKw(): number; // potencia pico
  getStatus(): NodeStatus; // ONLINE | PARTIAL | OFFLINE
  getChildCount(): number; // 0 si Leaf, N si Composite
  describe(): string; // árbol de texto
}
```

**Leaf** — un dispositivo físico individual:

```
class EnergyDevice implements IEnergyNode {
  getTotalKwh() → this.snap.energyKwh // su propio valor
  getPeakPowerKw() → this.snap.peakPowerKw // su propio valor
  getChildCount() → 0 // siempre cero
}
```

**Composite** — un grupo que contiene otros nodos:



```
class EnergyGroup implements IEnergyNode {
  private _children: IEnergyNode[];    // puede ser Leaf u otro EnergyGroup

  // RECURSIVIDAD - delega a cada hijo, sin saber si es Leaf o Group
  getTotalKwh()    → this._children.reduce((sum, c) => sum + c.getTotalKwh(), 0)
  getPeakPowerKw() → this._children.reduce((sum, c) => sum + c.getPeakPowerKw(), 0)

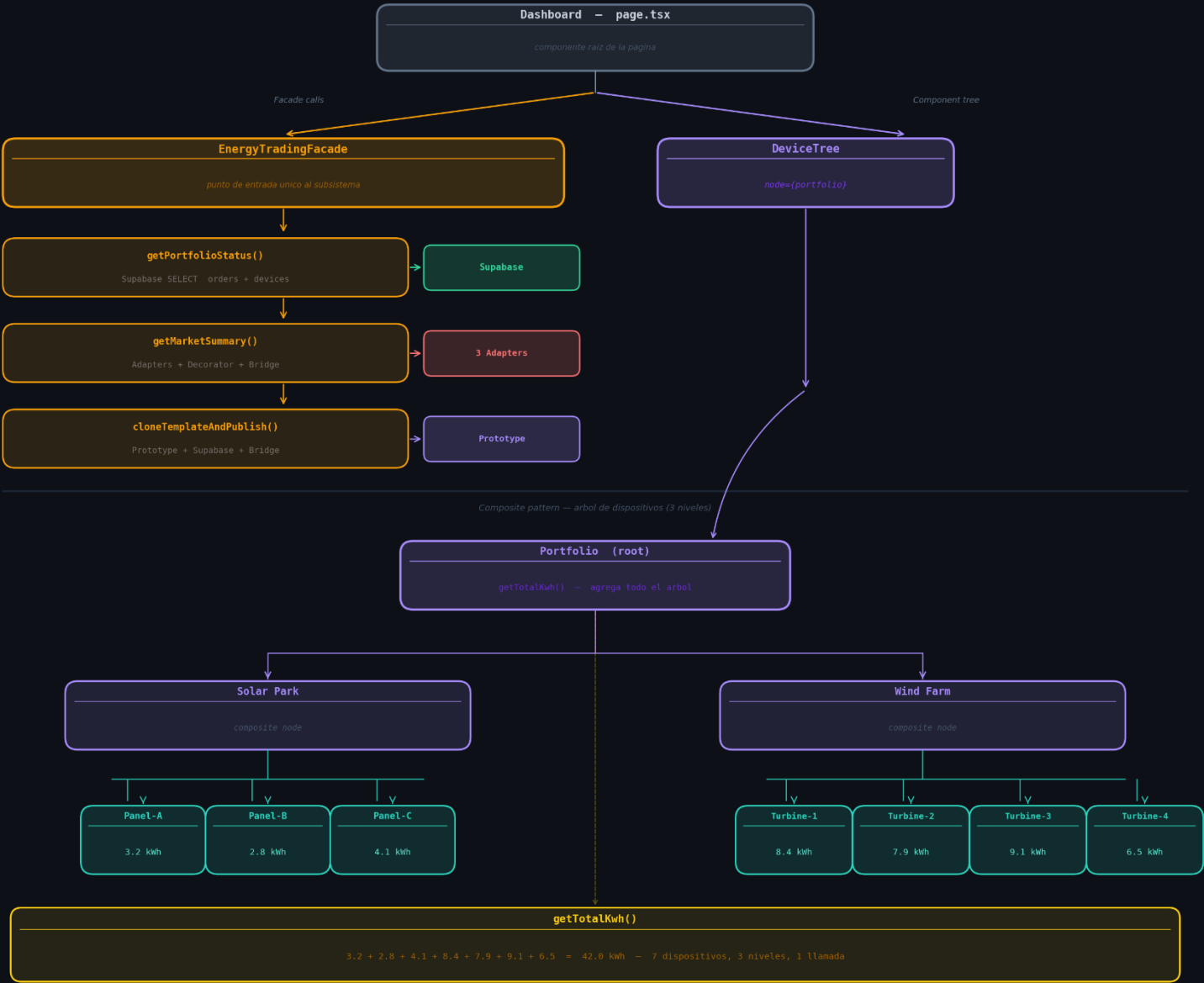
  // ONLINE si todos online; PARTIAL si alguno; OFFLINE si ninguno
  getStatus() → allOnline ? "ONLINE" : anyOnline ? "PARTIAL" : "OFFLINE"

  // Cuenta todos los nodos en el subárbol (recursivo)
  getChildCount() → children.reduce((sum, c) => sum + 1 + c.getChildCount(), 0)
}
```

Diagrama Uml

FACADE + COMPOSITE — Dashboard Architecture

page.tsx coordina Facade para operaciones y Composite para el arbol de dispositivos



CLAVE DEL DIAGRAMA

- page.tsx / Dashboard**  
componente React raiz, orquesta todo
- Supabase**  
PostgreSQL — INSERT ordenes / SELECT portfolio
- Composite (Portfolio)**  
arbol de dispositivos — leaf y branch misma interfaz
- EnergyTradingFacade**  
oculta complejidad: Builder, Adapters, Prototype, Bridge
- 3 Adapters + Decorator**  
OMIE, ENT50-E, Octopus con cache, log y retry
- Leaf devices (7 total)**  
Panel-A/B/C, Turbine-1/2/3/4 — cada uno retorna kWh